BASA: Ready2Work

Project Specification & Accelerated Implementation Plan

Project type Hackathon MVP	Delivery window 12 days or less
Platforms Android + Web	Core outcome Skills → Evidence → Work

BASA means Bridge to Actual Skills & Access. The name evokes lighting a fire; Ready2Work is the path from that spark to a real opportunity.

Purpose: give the team a practical product requirements document, technical blueprint, security baseline, GitHub workflow, testing strategy and prioritised 12-day delivery plan.

Prepared as a build guide for a team starting at the idea stage. Scope is deliberately focused on a reliable, demonstrable MVP.

Contents

1. Executive summary and product definition
2. Product Requirements Document (PRD)
3. MVP scope and acceptance criteria
4. System architecture and core workflows
5. Database schema and data relationships
6. Security, privacy and South African context
7. Matching engine specification
8. Screen and navigation specification
9. GitHub repository and collaboration workflow
10. Accelerated 12-day implementation plan
11. Prioritised sprint backlog and first tasks
12. Testing and quality plan
13. Success metrics and demo script
14. Risks, decisions and next actions

Appendix A. Suggested starter code and official references

Deadline adjustment

The original four-week roadmap has been compressed into a 12-day delivery plan because the submission is due in less than two weeks. This means AI, WhatsApp, transport payments, advanced mapping and full offline synchronisation are optional/deferred. The team should protect the end-to-end candidate-to-employer workflow and its security checks.

1. Executive summary and product definition

BASA: Ready2Work is a skills-first, low-data employment platform designed to turn informal experience into credible evidence and connect young South Africans to suitable entry-level jobs, paid tasks and short work opportunities.

The core product journey is: a candidate records a skill and evidence; BASA presents a transparent Skill Passport; the candidate discovers a relevant opportunity and applies; an employer reviews evidence and updates the application status.

The MVP must prove this journey end to end. It should not attempt to implement every long-term idea in the first release.

1.1 Problem statement

Many young people develop marketable abilities through informal work, volunteering, community activity, personal projects and small businesses. Their abilities can be difficult for employers to assess when they lack formal qualifications, CVs or references. Data costs, transport costs and limited connectivity create further barriers.

1.2 Product promise

- For candidates: make informal skills visible, provide evidence, find relevant work and track applications.
- For employers: discover candidates using relevant skill evidence rather than relying only on CVs or referrals.
- For assessors: use clear rubrics and record evidence-based outcomes.

1.3 MVP principles

- Skills-first, not credentials-only.
- Evidence labels must be honest: self-reported is not the same as verified.
- Matching must be explainable and must not make hiring decisions automatically.
- Collect minimal personal data and use broad location areas by default.
- Build one complete workflow before adding integrations.

1.4 Scope boundary

MVP — must deliver	Defer unless the core works
Authentication and roles; candidate profile; skill claims and evidence; Skill Passport; employer profile; job posting; rules-based matching; applications; employer shortlisting; essential admin/reporting; Android and web builds.	AI-generated CVs; WhatsApp bot; transport stipend disbursement; complex maps; advanced mentor chat; full offline sync; payroll/escrow; machine-learning ranking; large-scale credential integrations.

1.5 Definition of success

A judge or test user can register or sign in, create a candidate profile, record a skill, view its evidence status, find an open job, apply, and then see the employer review and update that application. Key permissions are tested, and the web/mobile demo is repeatable.

2. Product Requirements Document (PRD)

2.1 Target users

Persona	Needs	Success
Candidate	Represent informal skills, add evidence, find work, track applications.	Can show relevant ability and apply to an opportunity.
Employer	Post a genuine job, inspect evidence, shortlist applicants.	Can identify relevant candidates with less screening effort.
Administrator / assessor	Review reports and assess evidence consistently.	Can manage unsafe listings and record traceable assessment decisions.

2.2 Functional requirements

ID	Requirement	Priority	Acceptance criteria
FR-01	Authentication	P0	Register, sign in and sign out work.
FR-02	Role-based access	P0	Candidate/employer/admin actions are restricted by trusted role data.
FR-03	Candidate profile	P0	Candidate can save and edit skills, experience summary, broad location and availability.
FR-04	Skill evidence	P0	Candidate can submit a claim and supported evidence or assessment.
FR-05	Verification states	P0	Self-reported, peer-endorsed and task-assessed states are distinct.
FR-06	Skill Passport	P0	Passport displays skills, evidence types and verification status.
FR-07	Employer profile	P0	Employer can create and edit a business profile.
FR-08	Job management	P0	Employer can create, view and close own jobs with clear pay terms.
FR-09	Matching	P0	Open jobs are ranked using documented, testable rules.
FR-10	Applications	P0	Candidate applies and sees status; duplicate applications are prevented.
FR-11	Shortlisting	P0	Employer sees applicants for own jobs and changes status through valid transitions.
FR-12	Basic moderation	P0	Authorised admin can review a reported listing or assessment.
FR-13	Offline-friendly drafts	P1	Profile draft can be stored locally and recovered.
FR-14	AI CV drafting	P1	Server-side generation creates editable text from user-provided facts only.
FR-15	In-app notifications	P1	User sees application status updates.
FR-16	Location-aware filters	P1	User can filter by broad area or supported distance estimate.
FR-17	WhatsApp access	P2	Later integration supports selected journeys.
FR-18	Transport support	P2	Later workflow handles eligibility, approval and disbursement records.

2.3 Non-functional requirements

Area	Requirement
Security	Private data and privileged actions are protected by server-side checks and tested rules.
Privacy	Collect only necessary data; explain what employers can see and let candidates control sharing.
Performance	Use small assets, pagination and measured low-bandwidth testing.
Accessibility	Readable text, clear labels, good contrast and keyboard-friendly web UI.
Reliability	Handle loading, empty and error states; avoid duplicate applications.
Maintainability	Feature-based modules, consistent naming, reviewed PRs and automated checks.
Explainability	Show why a job is suggested and distinguish evidence levels.
Testability	Keep matching, validation and status rules independently testable.

2.4 User stories

- US-01 — As a candidate, I want to register so I can access BASA.
- US-02 — As a candidate, I want to record informal experience so my skills are not limited to formal employment history.
- US-03 — As a candidate, I want to submit evidence so employers can assess my ability.
- US-04 — As a candidate, I want to see verification status so I know what employers can reasonably trust.
- US-05 — As a candidate, I want to find relevant opportunities and apply without re-entering the same information.
- US-06 — As a candidate, I want to track application status.
- US-07 — As an employer, I want to post an opportunity with responsibilities, dates, pay and required skills.
- US-08 — As an employer, I want to compare candidates' evidence against role requirements.
- US-09 — As an employer, I want to shortlist or reject applicants and record the outcome.
- US-10 — As an administrator, I want to review reported listings.
- US-11 — As an assessor, I want a consistent rubric and a record of my assessment.

2.5 Definition of done

- The complete user journey works and persists data.
- Authentication and authorisation are enforced, not just hidden in the UI.
- Loading, empty and error states are handled.
- Relevant automated tests pass.
- A pull request has been reviewed and merged.
- A manual test path is documented for the demo.

3. MVP scope and acceptance criteria

3.1 Required candidate flow

```
Open BASA -> Register / sign in -> Complete profile -> Add skill claim
-> Submit evidence / complete supported assessment -> View Skill Passport
-> Discover open jobs -> View match reasons -> Apply -> Track status
```

3.2 Required employer flow

```
Sign in -> Complete employer profile -> Create job with pay, dates and skills
-> Publish/open job -> View applicants for own job -> Inspect allowed evidence
-> Move application through valid status transitions
```

3.3 Skill verification labels

Label	Meaning	What must not be implied
Self-reported	Candidate entered the claim.	Not independently confirmed.
Peer-endorsed	An identified peer provided an endorsement.	Not equivalent to formal certification.
Task-assessed	A recorded assessment was completed against a rubric.	Not a universal guarantee of job performance.
Mentor-reviewed	A designated mentor reviewed submitted evidence.	Not necessarily an accredited qualification.

3.4 Required job information

- Title, responsibilities, required skill IDs, work mode and broad location.
- Start/end dates or expected duration, application deadline and job status.
- Compensation amount in ZAR and a clear unit: hourly, daily or fixed amount.
- Employer ownership and a way to report a suspicious or misleading listing.

3.5 Exclusions for the deadline

Do not block the release on OpenAI integration, WhatsApp, map-based routing, payment processing, complex chat or full offline synchronisation. These introduce extra security, operational and testing risks. Add them only if the P0 journey is stable.

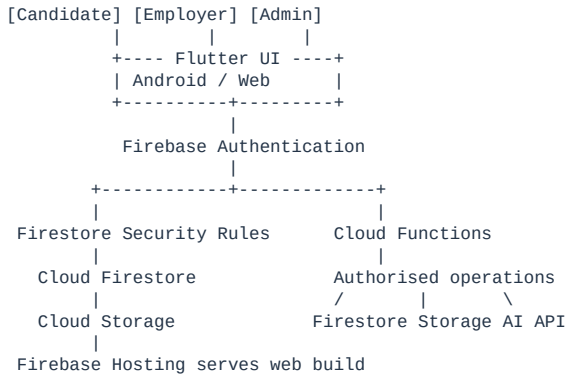
4. System architecture and core workflows

4.1 Recommended stack

Layer	Technology	Purpose
Client	Flutter / Dart	Shared Android and browser UI.
Identity	Firebase Authentication	Registration and sign-in.
Database	Cloud Firestore	Profiles, claims, jobs and applications.
File storage	Cloud Storage for Firebase	Optional portfolio evidence with restricted access.
Privileged operations	Cloud Functions	Role checks, state transitions and optional AI calls.
AI (P1)	OpenAI API via server only	Draft CV/profile wording from candidate-provided facts.
Hosting	Firebase Hosting	Publish browser app.
Collaboration	GitHub + Figma	Source control, issues, reviews and UI design.

Recommendation: use one Flutter codebase for Android and browser delivery in this deadline. A shared Flutter web app is not automatically a minimal-download PWA; measure initial bundle size. If it is too heavy, improve asset loading and defer a separate text-first PWA until after the submission.

4.2 Architecture diagram



Security note: Security Rules protect direct client access to Firebase resources. Server-side code using privileged Admin SDK credentials must perform its own authentication/authorisation checks. Hiding a button in the UI is not security.

4.3 Core workflow diagram

```
Candidate: profile -> skill claim -> evidence -> assessment -> passport
    -> discover job -> apply -> track status

Employer: profile -> publish job -> review applicants -> shortlist / reject
    -> select -> engagement occurs -> completion is confirmed
```

4.4 Role-based navigation

- Candidate: Home, Skills, Opportunities, Applications, Profile.
- Employer: Dashboard, My Jobs, Applicants, Business Profile.
- Admin: Overview, Reports, Assessments.

4.5 Suggested Flutter structure

```
app/lib/
  app/
  core/config/
  core/errors/
  core/theme/
  core/widgets/
  features/auth/
  features/candidate/
  features/skills/
  features/passport/
  features/jobs/
  features/applications/
  features/employer/
  features/admin/
  models/
  repositories/
  main.dart
app/test/
functions/src/
firebase/firestore.rules
firebase/firestore.indexes.json
firebase/storage.rules
docs/
```

5. Database schema and data relationships

Firestore is a document database; the collection layout below is a conceptual schema. The application must validate referenced IDs and ownership because Firestore does not automatically enforce foreign-key

relationships.

```
users/{uid}
  displayName, email, role, createdAt, accountStatus

candidateProfiles/{uid}
  headline, experienceSummary, skillIds, locationArea,
  availability, updatedAt

employerProfiles/{uid}
  businessName, description, verificationStatus, createdAt

skills/{skillId}
  name, category, assessmentTemplateId, active

skillClaims/{claimId}
  candidateId, skillId, selfReportedLevel, evidenceType,
  evidencePath, verificationStatus, createdAt

assessments/{assessmentId}
  candidateId, skillId, rubricVersion, score, assessorId,
  decision, feedback, createdAt

skillPassports/{passportId}
  candidateId, version, summary, publishedAt, visibility

jobs/{jobId}
  employerId, title, description, requiredSkillIds,
  locationArea, workMode, compensationAmount,
  compensationCurrency, compensationUnit, startDate,
  endDate, applicationDeadline, status

applications/{applicationId}
  jobId, candidateId, status, submittedAt, updatedAt

workRecords/{workRecordId}
  jobId, candidateId, employerId, completionStatus,
  employerConfirmed, feedback

reports/{reportId}
  reporterId, targetType, targetId, reason, status, createdAt
```

5.1 Conceptual relationships

```
USER 1--0..1 CANDIDATE_PROFILE
USER 1--0..1 EMPLOYER_PROFILE
CANDIDATE_PROFILE 1--many SKILL_CLAIM
SKILL 1--many SKILL_CLAIM
SKILL_CLAIM 1--many ASSESSMENT
CANDIDATE_PROFILE 1--many SKILL_PASSPORT
EMPLOYER_PROFILE 1--many JOB
JOB 1--many APPLICATION
CANDIDATE_PROFILE 1--many APPLICATION
APPLICATION 1--0..1 WORK_RECORD
```

5.2 Data design decisions

- Use Firebase Authentication UIDs as identity references.
- Assign privileged roles only through trusted server-side processes.
- Keep candidate claims separate from assessor decisions.
- Every job has an owner; employers only access applications for their own jobs.
- Prevent duplicate applications with a deterministic application ID or trusted transactional check.
- Use explicit ZAR compensation units and clear payment terms.
- Passport visibility is explicit. Never expose email, private applications or raw evidence by default.
- Define account deletion, correction and evidence-retention behaviour before public launch.

6. Security, privacy and South African context

6.1 Access-control matrix

Action	Candidate	Employer	Admin / assessor
Read open job listings	Yes	Yes	Yes
Edit own profile	Own profile	Own business profile	Limited, authorised
Submit skill evidence	Own claims	No	Only where assigned
Change assessment result	No	No	Authorised assessors only
Publish/edit jobs	No	Own permitted jobs	Moderation only
Read all applications	No	No	Only if necessary and authorised
Read applications to own jobs	No	Yes	Restricted oversight
Change user role	No	No	Trusted administrative process

6.2 Security baseline

- Require authentication for private operations.
- Validate field types, allowed values and state transitions.
- Enforce ownership and role permissions in Security Rules and server-side code.
- Prevent candidates from editing assessor IDs, assessment scores and verification labels.
- Prevent employers from accessing other employers' applicants.
- Restrict uploaded evidence by type, size and access permissions.
- Keep API secrets out of client builds, GitHub and logs.
- Use rate limits or abuse controls for privileged operations where appropriate.
- Test Firestore and Storage rules with the Firebase Emulator Suite before deployment.
- Seed demo data with fictional people and opportunities.

6.3 POPIA-aware product choices

BASA will process personal information such as names, contact details, skills and application information. Design for the Protection of Personal Information Act (POPIA):

- Collect only information needed for the service.
- Use broad location areas rather than precise GPS by default.
- Explain profile visibility and obtain appropriate permission before sharing candidate information.
- Do not collect identity documents without a justified need.
- Provide a process for correcting information and handling deletion requests.
- Use real participant data only with appropriate notice, consent/other lawful basis and safeguards.

This is a product checklist, not legal advice or a substitute for a compliance review before public launch.

7. Matching engine specification

Start with a rules-based scoring function rather than machine learning. It is cheaper to build, easier to test and easier to explain to candidates and employers.

7.1 Initial score

$$M = 0.50S + 0.25E + 0.15A + 0.10L$$

- S = required-skill alignment (0–100).
- E = strength and relevance of evidence (0–100).
- A = availability fit (0–100).
- L = location/work-mode fit (0–100).

Weights are initial hypotheses, not validated predictors of job performance. Treat the score as a recommendation aid, not a hiring decision.

7.2 Matching rules

- Exclude closed jobs and genuine, necessary eligibility mismatches.
- Calculate skill alignment against required skills.
- Evaluate evidence using explicit states and role relevance.
- Check availability, broad location and remote-work fit.
- Sort by score and show the main reasons for the recommendation.
- Do not treat unverified skills as absent; show candidates what evidence could improve their profile.
- Do not use sensitive characteristics to rank candidates.

7.3 Implementation and tests

Implement a small pure Dart service/function. Unit-test perfect match, missing required skills, incomplete profile, remote job, unavailable candidate, closed job and ties. Keep the scoring weights in one place and document changes.

8. Screen and navigation specification

8.1 Candidate screens

Screen	Required content
C-01 Candidate dashboard	Profile completion, recommended jobs, application updates and next step.
C-02 Skills and evidence	Add skill, describe experience, submit optional evidence and view status.
C-03 Skill Passport	Skill claims, verification method, assessment status and shareable information.
C-04 Opportunity detail	Responsibilities, pay, dates, work mode, required skills, employer and apply action.
C-05 Applications	Submitted applications, current status and relevant updates.

8.2 Employer screens

Screen	Required content
E-01 Employer dashboard	Active jobs, new applications and shortlisted candidates.
E-02 Create opportunity	Title, description, skills, dates, compensation and work mode.
E-03 Applicant list	Relevant skills, evidence labels and status.
E-04 Candidate detail	Allowed profile fields, supporting evidence and decision controls.

8.3 Admin screens

- Review reported opportunities.
- Review assessment records and record decisions.
- Flag/suspend suspicious listings through a trusted backend action.
- View a small set of aggregate pilot metrics.

Only build screens that support a user story or a necessary operational workflow. Ensure all forms have validation and clear loading, empty and error states.

9. GitHub repository and collaboration workflow

9.1 Repository layout

```
basa-ready2work/
  README.md
  CONTRIBUTING.md
  docs/
    PRD.md
    architecture.md
    database-schema.md
    security.md
    testing-plan.md
  app/
    lib/
    test/
    web/
    android/
  functions/
    src/
    test/
  firebase/
    firestore.rules
    firestore.indexes.json
    storage.rules
  .github/workflows/ci.yml
```

9.2 Branching

- main: always demonstrable; no direct feature commits.
- feature/short-name: one branch per issue.
- Open a pull request, run checks, request review, then merge.

```
git clone <repository-url>
cd basa-ready2work
git switch -c feature/candidate-profile

# Implement and test the assigned task
git add .
git commit -m "feat: add candidate profile"
git push -u origin feature/candidate-profile
```

9.3 Commit prefixes

Prefix	Example
feat	feat: add skill evidence submission
fix	fix: prevent duplicate applications
test	test: cover matching score calculation
docs	docs: add database schema
refactor	refactor: separate job repository
chore	chore: configure Firebase emulator

9.4 Pull request checklist

- Feature satisfies linked issue acceptance criteria.
- Loading, empty and error states are handled.
- Access permissions considered and tested.
- Relevant automated tests pass.
- Documentation is updated where needed.
- Screenshots or a demo recording are attached for UI changes.
- Known limitations and follow-up work are documented.

9.5 CI starter workflow

```
name: BASA CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  flutter-checks:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: app
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
      - run: flutter pub get
      - run: dart format --output=none --set-exit-if-changed lib test
      - run: flutter analyze
      - run: flutter test
```

This is a starter example, not a validated production workflow. Pin third-party actions to reviewed commit SHAs before production use. Add backend and Firebase Security Rules tests once those components exist. Never put production secrets in the repository or PR logs.

10. Accelerated 12-day implementation plan

The deadline is less than two weeks, so this plan compresses the original four-week roadmap. Use daily demos and a strict feature freeze. If a feature threatens the core workflow, defer it.

Day	Focus	Deliverable / exit criterion
1	Scope and team	Agree on P0 flow, roles, success criteria, repo and task owners.
2	User flows and UI	Figma wireframes, screen list, field definitions and navigation.
3	Architecture and schema	Approve collections, role matrix, application states and security approach.
4	Project setup	Flutter app runs on web and Android; Firebase dev environment configured.
5	Authentication	Register/sign-in/sign-out and trusted role routing work.
6	Candidate profile	Profile create/edit persists and has validation and error states.
7	Skills and passport	Skill claim can be added and displayed with honest status labels.
8	Employer and jobs	Employer profile and job creation/closure work.
9	Applications	Candidate applies; duplicate prevention and status tracking work.
10	Matching and employer review	Explainable score and employer shortlist/reject workflow work.
11	Security and regression	Rules, permission, workflow and device/browser tests; fix P0 defects.
12	Deployment and demo	Seed data, deploy web build, prepare Android build and rehearse presentation.

Daily cadence

- Morning: confirm the day's one demonstrable outcome and blockers.
- Midday: integrate branches early; resolve shared-model conflicts.
- End of day: demo the current main branch and update issue statuses.

Scope cut-off

- If behind on Day 6: defer AI, maps, chat and offline drafts.

- If behind on Day 9: protect applications and security; cut secondary dashboards and visual polish.
- Do not remove access controls or the candidate-to-employer journey to make the demo appear finished.

11. Prioritised backlog and concrete first tasks

ID	Issue	Owner	Acceptance criteria
BASA-001	Create repository and docs	Product lead / dev	README, PRD, architecture, project board and labels exist.
BASA-002	Create Flutter Android/web app	Frontend	App runs in browser and on Android target.
BASA-003	Configure Firebase dev	Backend	Auth and Firestore configured; no secrets committed.
BASA-004	Core models and enums	Both	Profile, SkillClaim, Job, Application models and tests.
BASA-005	Authentication and role routing	Both	Register/sign-in/sign-out; routes use trusted role.
BASA-006	Candidate profile	Frontend	Create/edit, persist and handle loading/error.
BASA-007	Skill claims and passport	Both	Claims visible with status; candidate cannot change results.
BASA-008	Jobs and applications	Both	Create/close job, apply, enforce ownership and permissions.
BASA-009	Matching engine	Backend	Explainable scores; edge cases and closed-job filtering tested.
BASA-010	Demo and regression	QA / product	Seed fictional data; full flow works; defects and fallback documented.

First working session

1. Create the GitHub repository named basa-ready2work and set up Backlog, Ready, In Progress, Review, Testing and Done columns.

2. Assign feature owners. One person can hold multiple roles on a small team, but every issue needs one accountable owner.

3. Freeze the P0 journey and choose one or two skill categories to demonstrate credibly.

4. Sketch the main screens and agree on Firestore fields, role permissions and verification states.

5. Build a vertical slice: register a candidate, save a profile, add a skill claim, show the Skill Passport, then sign out and back in to verify persistence.

Starter command

```
flutter create --platforms=android,web app
cd app
flutter run -d chrome
```

Assumes Flutter and Chrome are installed. If the repository already contains a Flutter project, add/enable web support rather than recreating it. Follow the official Flutter web setup and deployment documentation.

12. Testing and quality plan

12.1 Test levels

Level	What to test	Examples
Unit	Pure business rules and models	Matching score, validation, status transitions.
Widget	Forms, screen states and navigation	Invalid form, empty job list, error display.
Integration	Firebase-backed workflows	Profile save, job post, application create.
Security	Ownership and permission boundaries	Candidate cannot change score; employer cannot read another employer's applicants.
End-to-end	Full user journey	Candidate applies; employer reviews and changes status.

12.2 Minimum test cases

ID	Scenario	Expected result
T-01	Candidate registers	Account created with correct role.
T-02	Candidate edits own profile	Data persists and can be retrieved.
T-03	Candidate changes assessment result	Operation denied.
T-04	Employer posts a job	Open job appears in listings.
T-05	Candidate applies twice	Duplicate application prevented.
T-06	Employer reads another employer's applicants	Access denied.
T-07	Candidate applies to closed job	Application rejected.
T-08	Matcher receives missing optional skills	No crash; documented result.
T-09	Connectivity lost during supported draft	Local draft remains recoverable.
T-10	AI asked to invent qualifications	Output constrained to supplied facts and reviewed before saving.
T-11	Unauthorised admin access	Access denied.
T-12	Invalid status transition	Transition rejected.

12.3 Low-data measurement

- Measure initial web download size.
- Measure data used to open dashboard, browse 10 jobs, open passport and submit application.
- Test on a real phone and constrained connection if possible.
- Record device, network conditions and method; do not claim universal data savings.
- Test offline draft behaviour and connectivity recovery explicitly.

13. Success metrics and demonstration

13.1 Suggested pilot targets

These are starting targets for planning, not guaranteed outcomes.

Metric	Suggested initial target
Candidate profiles completed	20
Participating businesses	3
Genuine paid opportunities	3–5

Metric	Suggested initial target
Critical workflow tests	100% of defined critical cases pass
Unauthorised access tests	0 successful unauthorised accesses
End-to-end demo	Candidate-to-employer workflow completed successfully
User feedback	At least 5 sessions

Do not count an application or shortlist as an employment outcome. Record selection, work completion and payment confirmation separately where applicable.

13.2 Five-minute demo script

Time	Demo segment	Show
0:00–0:45	Problem	A fictional young person has real skills but little formal work history.
0:45–1:45	Skill Passport	Add a skill, provide evidence and show its honest assessment status.
1:45–2:45	Matching and application	Show a paid opportunity, explain match reasons and submit application.
2:45–4:00	Employer workflow	Switch account, review allowed evidence and shortlist candidate.
4:00–5:00	Innovation and impact	Explain low-data approach, access controls, limitations and pilot plan.

Suggested closing message

“BASA does not just help young people find job advertisements. It helps them demonstrate what they can do, gives employers better evidence to make decisions, and creates a pathway from informal skills to meaningful work.”

14. Risks, decisions and next actions

Risk	Mitigation
Scope grows beyond deadline	Freeze P0 after Day 2; defer integrations.
Security rules become a late blocker	Create access matrix early and test with Firebase Emulator Suite.
Conflicting code changes	Feature owners, small PRs, early integration and no direct commits to main.
AI integration takes too long	Deliver core journey without AI; only add if stable.
No real employer participation	Contact potential businesses early; use honest fictional demo data if needed.
Web app is heavy	Measure bundle size, compress assets and defer unnecessary dependencies.
Verification lacks credibility	Use clear evidence labels and consistent rubrics.
Demo fails	Seed fictional data, rehearse, and keep a fallback recording.
Fraud or non-payment risk	Employer review, transparent compensation terms, reporting and a documented pilot process.

Immediate action checklist

- Create the repository, README and project board.
- Assign an owner to each P0 feature and first issue.
- Agree on one or two initial skill categories and the first assessment rubric.
- Confirm user stories, Firestore schema and role permissions.
- Get the app running on browser and Android.
- Build and test the first vertical slice before starting optional features.

- Schedule a daily demo and freeze scope after the first two days.

Final delivery principle

Build the smallest version that proves the biggest idea: informal skills can become credible evidence, credible evidence can improve matching, and better matching can create a practical route into paid work.

Appendix A. Starter code and official references

A.1 Example application status values

```
submitted
under_review
shortlisted
selected
not_selected
completed
withdrawn
```

Only permit explicitly defined transitions. Selection is not proof that work was completed; create a work record and confirm completion separately.

A.2 Example matching score

M = 0.50S + 0.25E + 0.15A + 0.10L, where S is skill alignment, E is evidence strength, A is availability fit and L is location/work-mode fit. Each input is 0–100. These are provisional weights and should be reviewed after user testing.

A.3 Official documentation

Use official docs for current setup details; product SDKs and CI actions may change.

Flutter: Web setup

Flutter: Web deployment

Firebase: Flutter setup

Firebase: Firestore Security Rules

Firebase App Check

South African Government: POPIA

This document is a hackathon implementation specification, not a guarantee of regulatory compliance or employment outcomes. Before a public pilot, review privacy, security, payment terms, employer verification and operational responsibilities.